

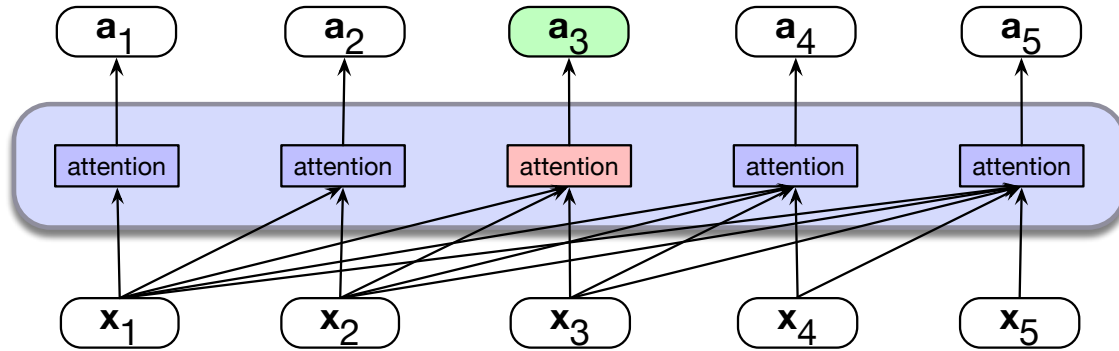
Masked Language Models

BERT

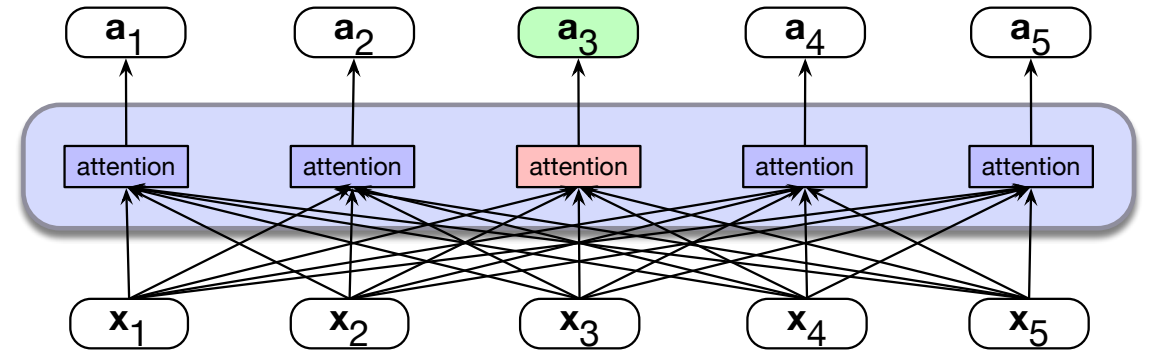
Masked Language Modeling

- We've seen autoregressive (causal, left-to-right) LMs.
- But what about tasks for which we want to peak at future tokens?
 - Especially true for tasks where we map each input token to an output token
- **Bidirectional** encoders use **masked self-attention** to
 - map sequences of input embeddings $(x_1, \dots, x_n)$
 - to sequences of output embeddings of the same length $(h_1, \dots, h_n)$,
 - where the output vectors have been contextualized using information from the entire input sequence.

Bidirectional Self-Attention



a) A causal self-attention layer



b) A bidirectional self-attention layer

Easy! We just remove the mask

Casual self-attention

N

$q1 \cdot k1$	$-\infty$	$-\infty$	$-\infty$
$q2 \cdot k1$	$q2 \cdot k2$	$-\infty$	$-\infty$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$-\infty$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

$$\mathbf{head} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Bidirectional self-attention

N

$q1 \cdot k1$	$q1 \cdot k2$	$q1 \cdot k3$	$q1 \cdot k4$
$q2 \cdot k1$	$q2 \cdot k2$	$q2 \cdot k3$	$q2 \cdot k4$
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$q3 \cdot k4$
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$

N

$$\mathbf{head} = \text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \mathbf{V}$$

BERT: Bidirectional Encoder Representations from Transformers

BERT (Devlin et al., 2019)

- 30,000 English-only tokens (WordPiece tokenizer)
- Input context window $N=512$ tokens, and model dimensionality $d=768$
- $L=12$ layers of transformer blocks, each with $A=12$ (bidirectional) multihead-attention layers.
- The resulting model has about 100M parameters.

XLM-RoBERTa (Conneau et al., 2020)

- 250,000 multilingual tokens (SentencePiece Unigram LM tokenizer)
- Input context window $N=512$ tokens, model dimensionality $d=1024$
- $L=24$ layers of transformer blocks, with $A=16$ multihead attention layers each
- The resulting model has about 550M parameters.

Masked Language Models

BERT

Masked Language Models

Masked LM training

Masked training intuition

- For **left-to-right LMs**, the model tries to predict the last word from prior words:

The water of Walden Pond is so beautifully _____

- And we train it to improve its predictions.

- For **bidirectional masked LMs**, the model tries to predict one or more words from all the rest of the words:

The _____ of Walden Pond _____ so beautifully blue

- The model generates a probability distribution over the vocabulary for each missing token
- We use the cross-entropy loss from each of the model's predictions to drive the learning process.

MLM training in BERT

15% of the tokens are randomly chosen to be part of the masking

Example: "Lunch was **delicious**", if delicious was randomly chosen:

Three possibilities:

1. 80%: Token is replaced with special token [MASK]

Lunch was **delicious** -> Lunch was **[MASK]**

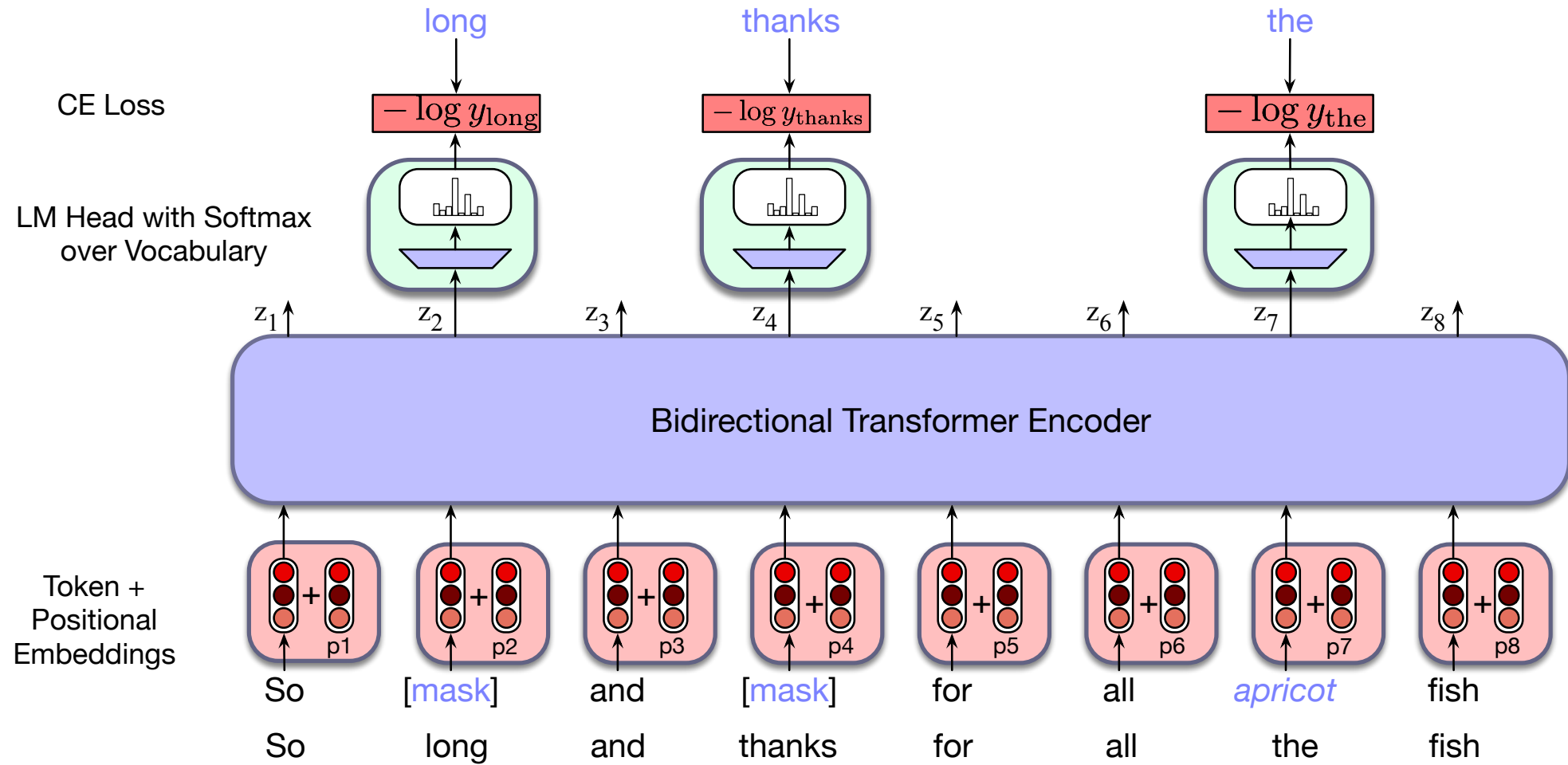
2. 10%: Token is replaced with a random token (sampled from unigram prob)

Lunch was **delicious** -> Lunch was **gasp**

3. 10%: Token is unchanged

Lunch was **delicious** -> Lunch was **delicious**

In detail



MLM loss

The LM head takes output of final transformer layer L , multiplies it by unembedding layer and turns into probabilities:

$$\mathbf{u}_i = \mathbf{h}_i^L \mathbf{E}^T$$

$$\mathbf{y}_i = \text{softmax}(\mathbf{u}_i)$$

E.g., for the x_i corresponding to "long", the loss is the probability of the correct word *long*, given output $\mathbf{h}_i^L$):

$$L_{MLM}(x_i) = -\log P(x_i | \mathbf{h}_i^L)$$

We get the gradients by taking the average of this loss over the batch

$$L_{MLM} = -\frac{1}{|M|} \sum_{i \in M} \log P(x_i | \mathbf{h}_i^L)$$

Next Sentence Prediction

Given 2 sentences the model predicts if they are a real pair of adjacent sentences from the training corpus or a pair of unrelated sentences.

BERT introduces two special tokens

- [CLS] is prepended to the input sentence pair,
- [SEP] is placed between the sentences, and also after second sentence

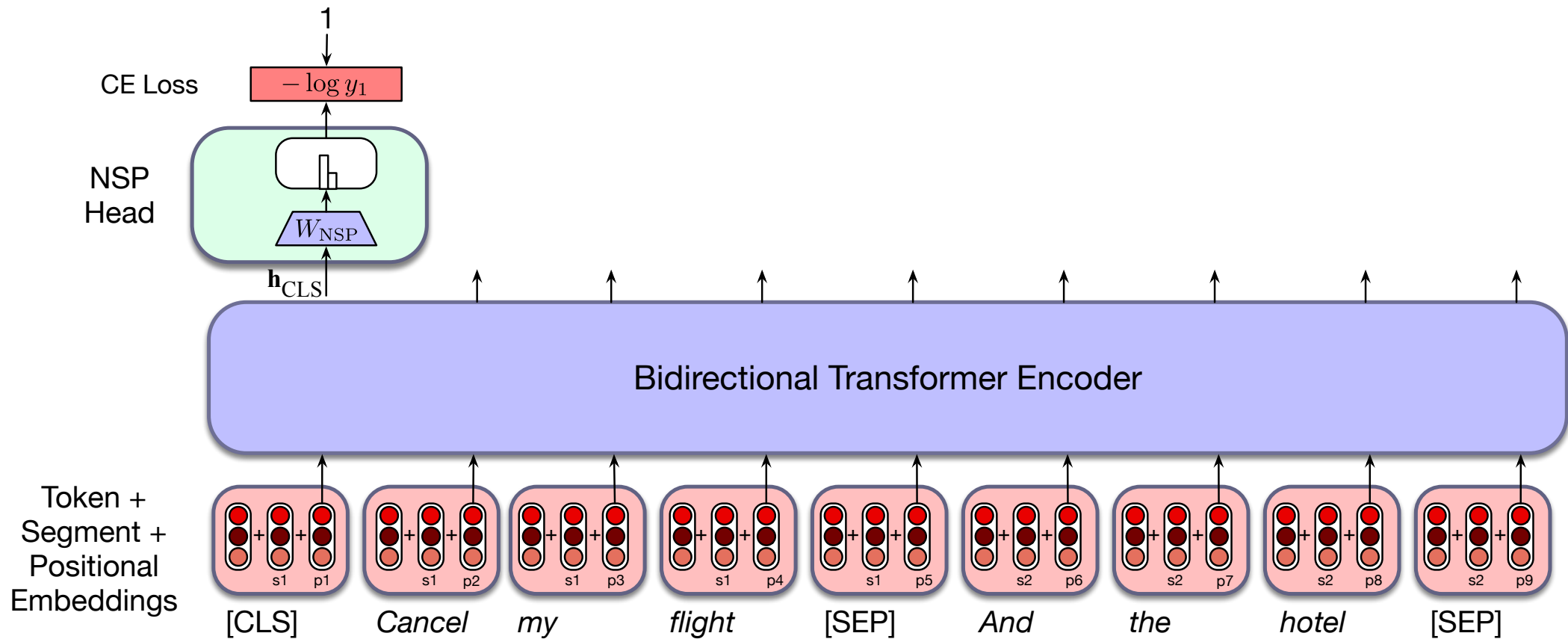
And two more special tokens

- [1st segment] and [2nd segment]
- These are added to the input embedding and positional embedding

h_{CLS}^L from the final layer [CLS] token is input to classifier head (weights W_{NSP}) that predicts two classes:.

$$\mathbf{y}_i = \text{softmax}(\mathbf{h}_{\text{CLS}}^L \mathbf{W}_{\text{NSP}})$$

NSP Loss with classification head



More details

Original model was trained with 40 passes over training data

Some models (like RoBERTa) drop NSP loss

Tokenizer for multilingual models is trained from stratified sample of languages (some data from each language)

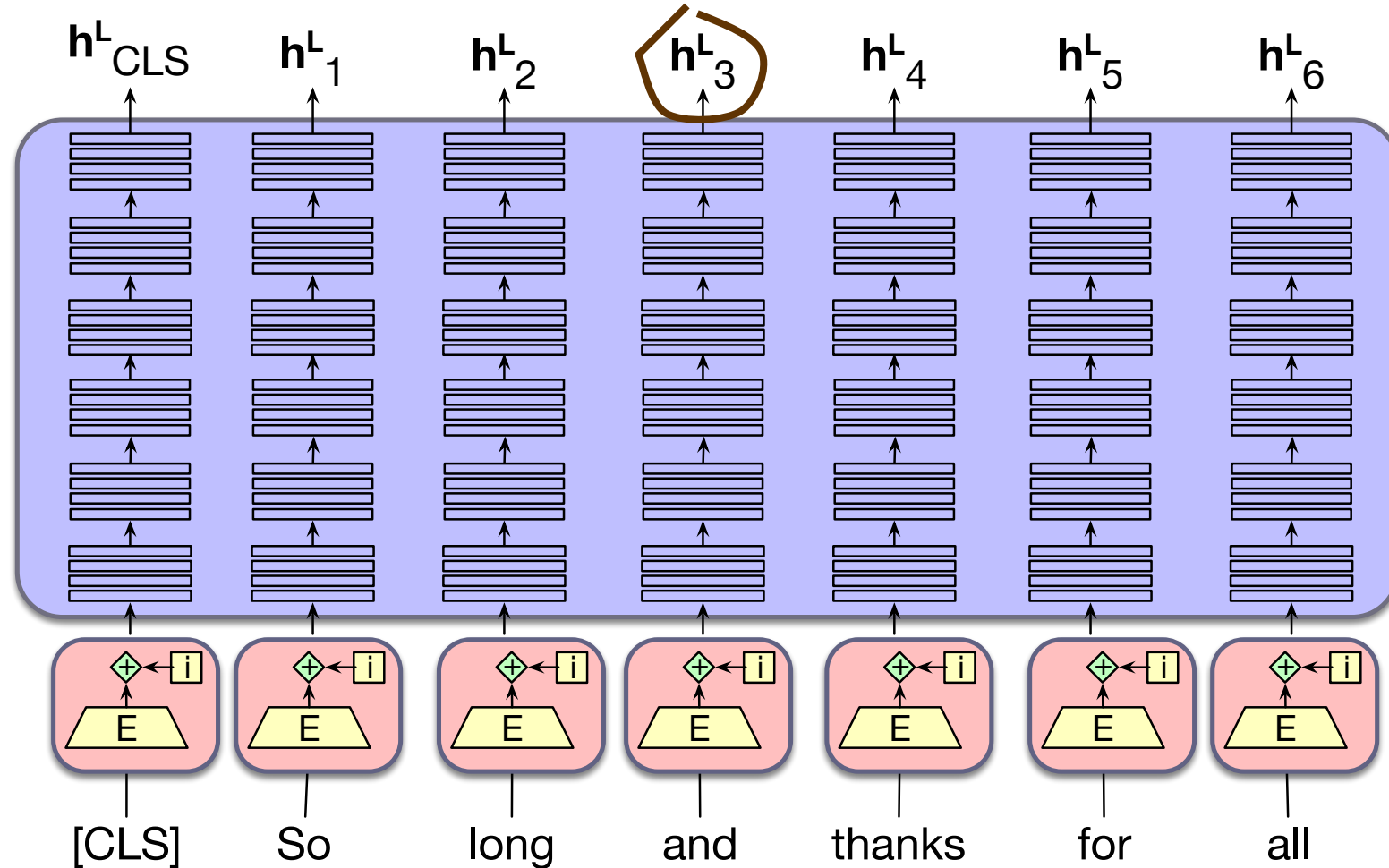
Masked Language Models

Masked LM training

Masked
Language
Models

Contextual Embeddings

Contextual Embeddings to represent words



Static vs Contextual Embeddings

- Static embeddings represent **word types** (dictionary entries)
- Contextual embeddings represent **word instances** (one for each time the word occurs in any context/sentence)



Word sense

Words are ambiguous

A **word sense** is a discrete representation of one aspect of meaning

mouse¹ : a *mouse* controlling a computer system in 1968.

mouse² : a quiet animal like a *mouse*

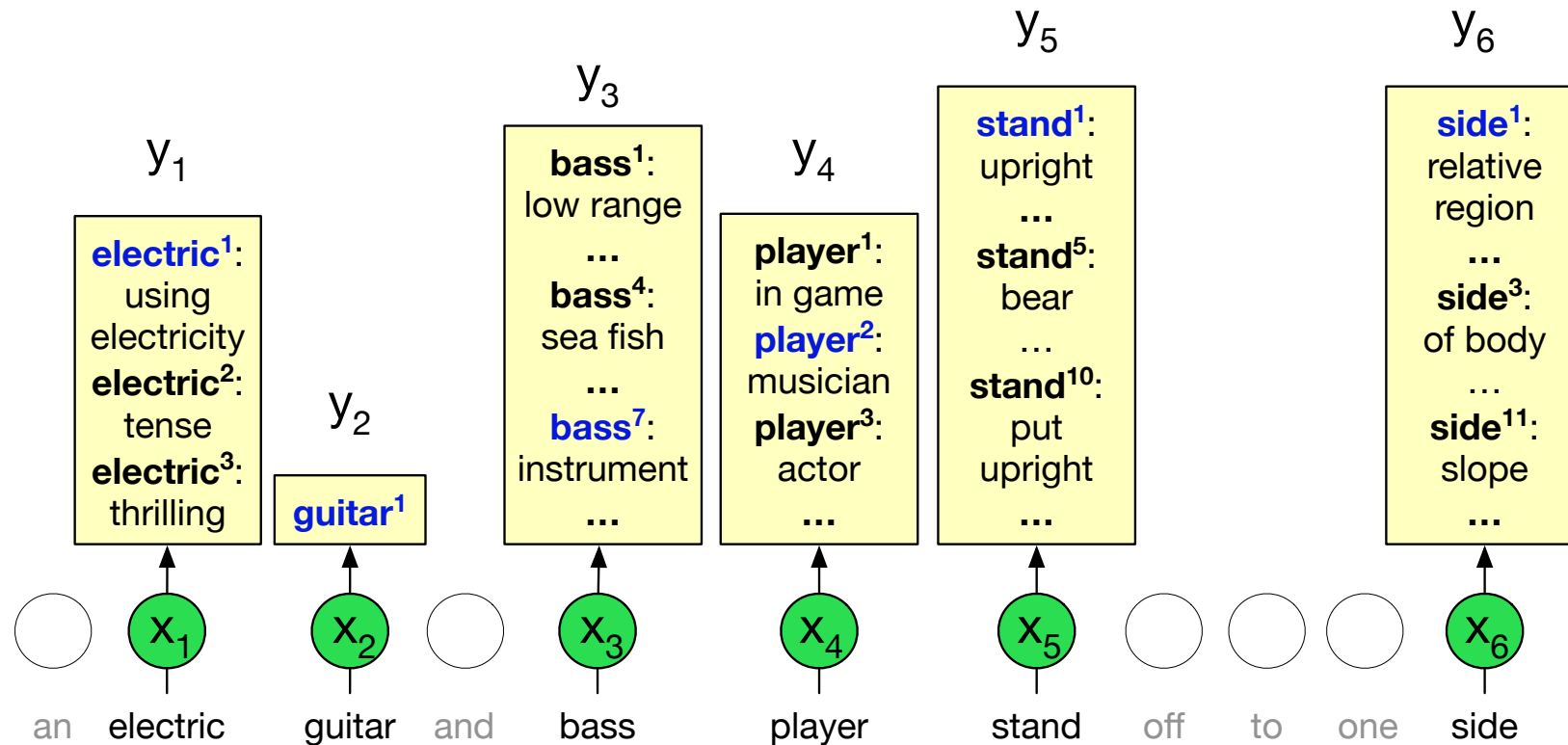
bank¹ : ...a *bank* can hold the investments in a custodial account ...

bank² : ...as agriculture burgeons on the east *bank*, the river ...

Contextual embeddings offer a continuous high-dimensional model of meaning that is more fine grained than discrete senses.

Word sense disambiguation (WSD)

The task of selecting the correct sense for a word.



1-nearest neighbor algorithm for WSD

Melamud et al (2016), Peters et al (2018)

At training time, take a sense-labeled corpus like SEMCOR

Run corpus through BERT to get contextual embedding for each token

- E.g., pooling representations from last 4 BERT transformer layer

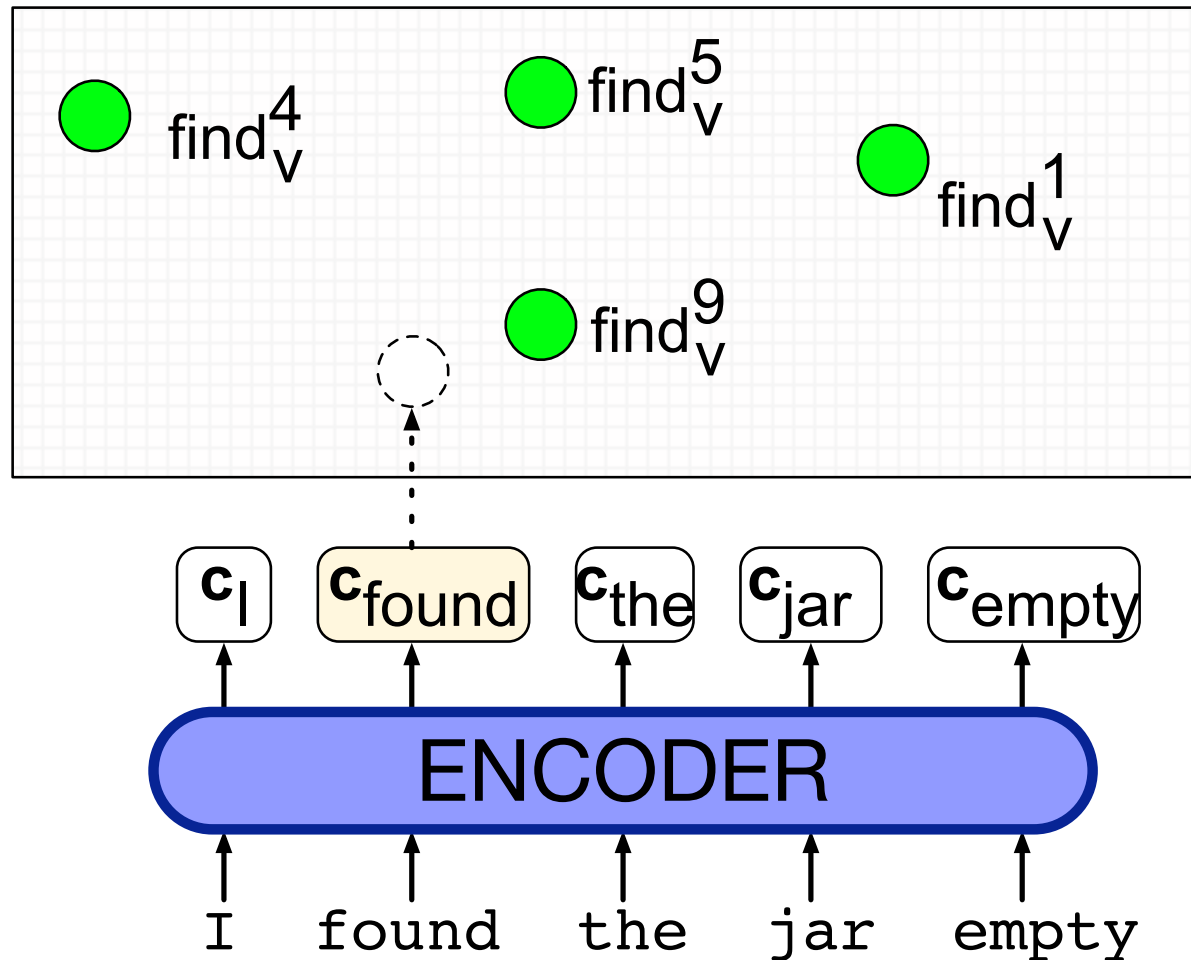
Then for each sense s of word w for n tokens of that sense, pool embeddings:

$$\mathbf{v}_s = \frac{1}{n} \sum_i \mathbf{v}_i \quad \forall \mathbf{v}_i \in \text{tokens}(s)$$

At test time, given a token of a target word t , compute contextual embedding $\mathbf{t}$ and choose its nearest neighbor sense from training set

$$\text{sense}(t) = \underset{s \in \text{senses}(t)}{\operatorname{argmax}} \cos(\mathbf{t}, \mathbf{v}_s)$$

1-nearest neighbor algorithm for WSD



Similarity and contextual embeddings

- In the context of **word embeddings**, **anisotropy** refers to the uneven distribution of word vectors in the high-dimensional embedding space. This means that some directions in the space are more "dense" or "preferred," which affects the way cosine similarity measures distances between embeddings.
- Ideally, vectors of semantically unrelated words should have low similarity (close to 0), while related words should have high similarity (close to 1).

~~A~~nisotropy Causes High Similarity for Unrelated Words

- Due to anisotropy, many word embeddings tend to cluster in a **narrow cone** rather than being uniformly distributed in space.
- As a result, even **random, unrelated words** may have a **high cosine similarity** just because they are in the same general direction in the space.

Dominant Directions in Embeddings

- Some high-dimensional embedding spaces have **a few dominant directions** that explain much of the variance.
- Words tend to align more in these directions, leading to **spurious correlations** (i.e., unrelated words having nonzero cosine similarity).

Effect on Word Similarity Computations

When comparing two word vectors using cosine similarity, the presence of **global anisotropy** may lead to:

- Overestimation of similarities for unrelated words.
- Reduced contrast between truly similar and dissimilar words.
- Poor performance in downstream NLP tasks like clustering and retrieval.

Mitigating A

Mean-Centering (Removing the Common Component)

Subtracting the **mean embedding vector** from all word embeddings reduces the effect of dominant directions.

Given a set C of all the embeddings in some corpus, each with dimensionality d (i.e., $\mathbf{x} \in \mathbb{R}^d$), the mean vector $\mu \in \mathbb{R}^d$ is:

$$\mu = \frac{1}{|C|} \sum_{\mathbf{x} \in C} \mathbf{x}$$

The standard deviation in each dimension $\sigma \in \mathbb{R}^d$ is:

$$\sigma = \sqrt{\frac{1}{|C|} \sum_{\mathbf{x} \in C} (\mathbf{x} - \mu)^2}$$

Then each word vector $\mathbf{x}$ is replaced by a standardized version $\mathbf{z}$:

$$\mathbf{z} = \frac{\mathbf{x} - \mu}{\sigma}$$

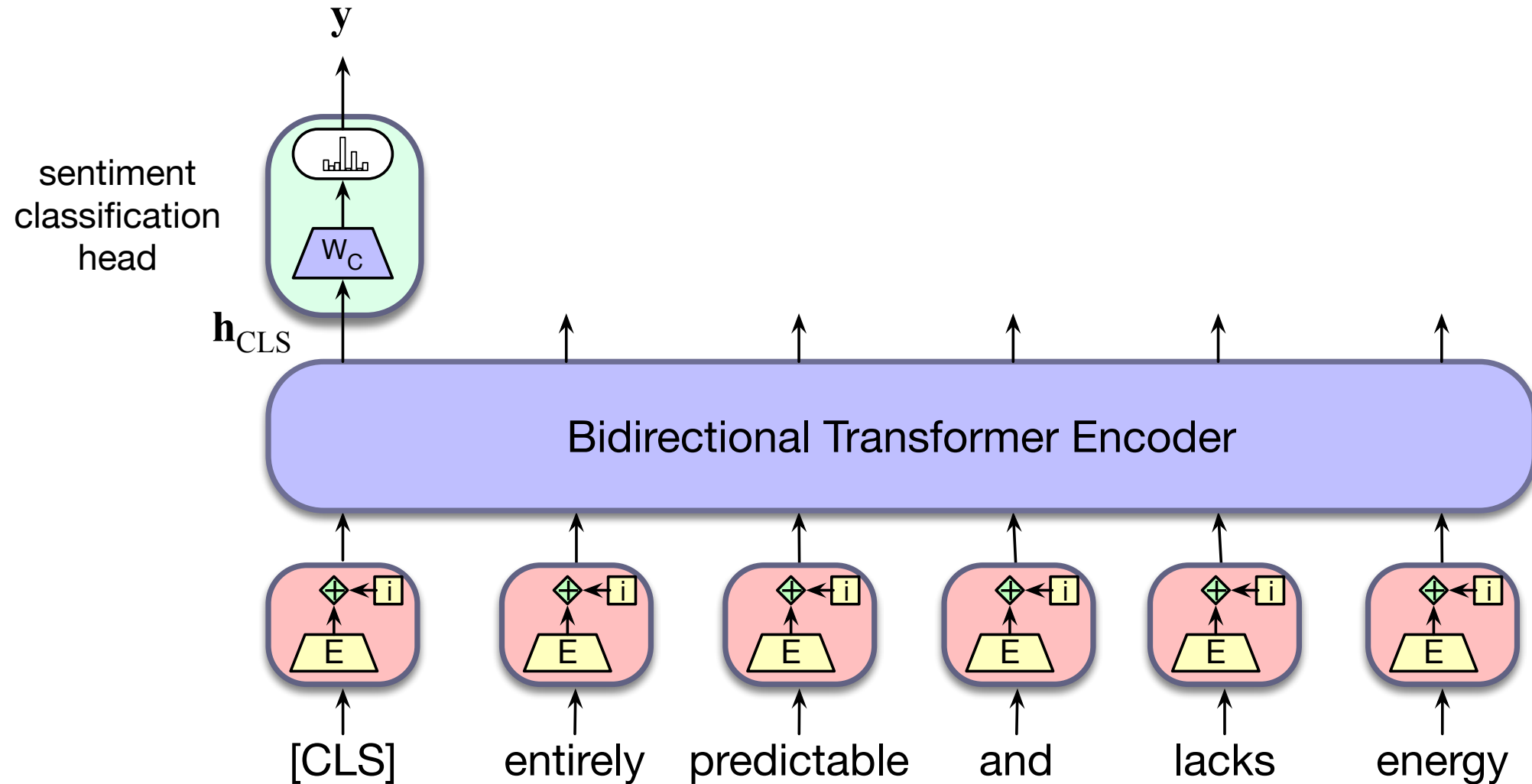
Masked
Language
Models

Contextual Embeddings

Masked
Language
Models

Fine-Tuning for Classification

Adding a sentiment classification head



Assuming a three-way sentiment classification task (positive, negative, neutral) and dimensionality d as the model dimension, W_C will be of size $[d \times 3]$.

To classify a document, we pass the input text through the pretrained language model to generate h_{CLS}^L , multiply it by W_C , and pass the resulting vector through a softmax.

$$\mathbf{y} = \text{softmax}(\mathbf{h}_{CLS}^L \mathbf{W}_C)$$

Finetuning the values in W_C requires supervised training data consisting of input sequences labeled with the appropriate sentiment class.

Training proceeds in the usual way; cross-entropy loss between the softmax output and the correct answer is used to drive the learning that produces W_C .

Sequence-Pair classification

Assign a label to pairs of sentences:

- paraphrase detection (are the two sentences paraphrases of each other?)
- logical entailment (does sentence A logically entail sentence B?)
- discourse coherence (how coherent is sentence B as a follow-on to sentence A?)

Example: Natural Language Inference

Pairs of sentences are given one of 3 labels

- Neutral
 - a: Jon walked back to the town to the smithy.
 - b: Jon traveled back to his hometown.
- Contradicts
 - a: Tourist Information offices can be very helpful.
 - b: Tourist Information offices are never of any help.
- Entails
 - a: I'm confused.
 - b: Not all of it is very clear to me.

Algorithm: pass the premise/hypothesis pairs through a bidirectional encoder and use the output vector for the [CLS] token as the input to the classification head .

Fine-tuning for sequence labeling

Assign a label from a small fixed set of labels to each token in the sequence.

- Named entity recognition
- Part of speech tagging
-

Named Entity Recognition

A **named entity** is anything that can be referred to with a proper name: a person, a location, an organization

Named entity recognition (NER): find spans of text that constitute proper names and tag the type of the entity

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	Turing is a giant of computer science.
Organization	ORG	companies, sports teams	The IPCC warned about the cyclone.
Location	LOC	regions, mountains, seas	Mt. Sanitas is in Sunshine Canyon .
Geo-Political Entity	GPE	countries, states	Palo Alto is raising the fees for parking.

Named Entity Recognition

Citing high fuel prices, [ORG United Airlines] said [TIME Friday] it has increased fares by [MONEY \$6] per round trip on flights to some cities also served by lower-cost carriers. [ORG American Airlines], a unit of [ORG AMR Corp.], immediately matched the move, spokesman [PER Tim Wagner] said. [ORG United], a unit of [ORG UAL Corp.], said the increase took effect [TIME Thursday] and applies to most routes where it competes against discount carriers, such as [LOC Chicago] to [LOC Dallas] and [LOC Denver] to [LOC San Francisco].

BIO Tagging

Ramshaw and Marcus (1995)

A method that lets us turn a segmentation task (finding boundaries of entities) into a classification task

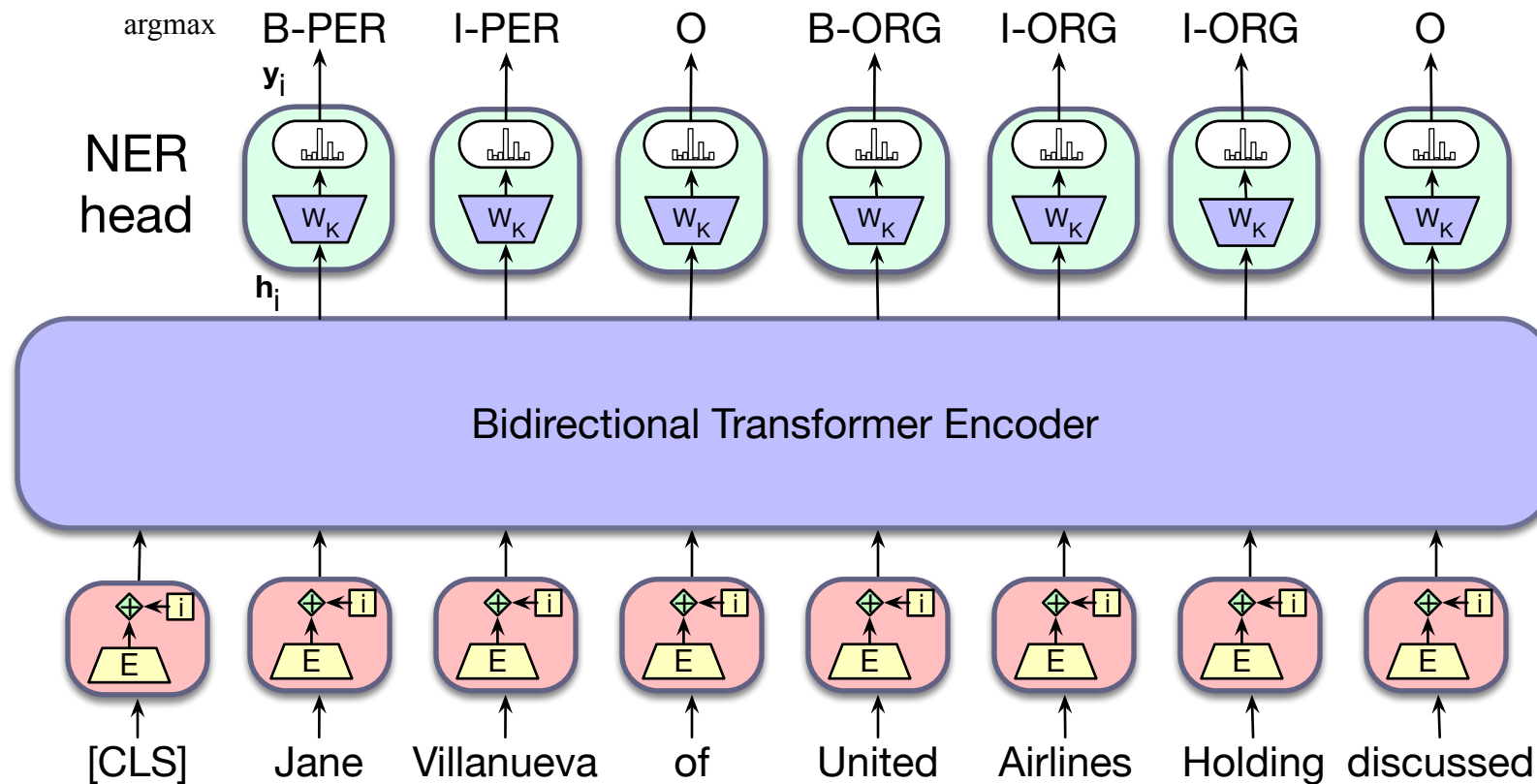
[**PER Jane Villanueva**] of [**ORG United**] , a unit of [**ORG United Airlines Holding**] , said the fare applies to the [**LOC Chicago**] route.

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

Sequence labeling

$$\mathbf{y}_i = \text{softmax}(\mathbf{h}_i^L \mathbf{W}_K)$$

$$\mathbf{t}_i = \text{argmax}_k(\mathbf{y}_i)$$



More details

We need to map between tokens (used by LLM) and words (used in definition of name entities)

We evaluate NER with F1 (precision/recall)

Masked
Language
Models

Fine-Tuning for Classification